

# Gerador de jogos educativos — Genius Factory

Especificação para gerar jogos em HTML (renderizados em iframe sandboxed) · versão revisada v2

Você vai gerar um jogo educativo em HTML completo (DOCTYPE + html + head + body) para a plataforma Genius Factory. O jogo será renderizado dentro de um iframe *sandboxed*. Leia o contrato abaixo: ele é a parte mais importante.

## 1. Contrato obrigatório

A plataforma só sabe que o aluno respondeu se o seu HTML executar esta chamada exata em algum momento:

```
window.parent.postMessage({
  tipo: 'resposta',
  acertou: true,           // ou false – boolean LITERAL, nunca string nem número
  meta: { ... }           // objeto padronizado (ver regra 8)
}, '*');
```

### Regras duras

1. O HTML deve conter pelo menos uma chamada literal a `window.parent.postMessage`. Sem isso, o jogo é inválido.
2. `acertou` deve ser boolean literal: `true` ou `false`. Nunca a string `"true"`, nunca o número `1`, nunca `undefined`.
3. O `postMessage` deve ser disparado tanto quando o aluno acerta quanto quando erra. Nunca deixe o aluno num estado "respondeu, mas o jogo não enviou nada".
4. Se o jogo permite várias tentativas internas (ex.: 3), envie o `postMessage` só na última (no sucesso ou na desistência final), nunca a cada tentativa.
5. Envie exatamente uma vez por jogo. Bloqueie cliques posteriores ao primeiro envio (use uma flag `respondido` no JS).
6. Mostre feedback visual dentro do jogo (cores, mensagens), mas isso não substitui o `postMessage` — são coisas independentes.
7. **NOVO Caminho terminal garantido.** Todo jogo precisa de pelo menos um caminho que *sempre* chega ao fim e dispara o `postMessage`: acerto, esgotamento das tentativas ou um botão de desistir. O aluno nunca pode ficar travado sem que nada seja enviado. Em jogos abertos (preenchimento, arrastar, montar), inclua sempre um botão de **Desistir** com confirmação que envia `acertou: false`.
8. **NOVO Objeto meta padronizado.** Para os relatórios do simulado saírem uniformes, use sempre as mesmas chaves quando fizer sentido: `tentativas` (número), `tempo` (segundos desde o início) e, quando aplicável, `resposta` (o que o aluno marcou ou montou).

## 2. Exemplo mínimo funcional

Use como base. Note o `postMessage` nos dois caminhos (acerto e erro) e a flag `respondido`:

```

<!DOCTYPE html>
<html lang="pt-BR">
<head><meta charset="UTF-8">
<style>
  body { font-family: sans-serif; padding: 20px; background: #faf9f5; text-align: center; }
  button { margin: 8px; padding: 12px 24px; font-size: 16px; border-radius: 8px;
    cursor: pointer; border: none; }
  .verde { background: #16a34a; color: #fff; } .vermelho { background: #dc2626; color:
#fff; }
  #fb { margin-top: 16px; font-weight: 600; }
</style></head>
<body>
  <h2>Clique no botão verde para acertar</h2>
  <button class="verde" id="b1">Verde</button>
  <button class="vermelho" id="b2">Vermelho</button>
  <div id="fb"></div>
  <script>
    var respondido = false, inicio = Date.now();
    function enviar(acertou) {
      if (respondido) return; respondido = true;
      document.getElementById('fb').textContent = acertou ? '\u2713 Correto!' : '\u2717
Errou.';
      window.parent.postMessage({ tipo: 'resposta', acertou: acertou,
        meta: { tempo: Math.round((Date.now()-inicio)/1000) } }, '*');
    }
    document.getElementById('b1').onclick = function(){ enviar(true); };
    document.getElementById('b2').onclick = function(){ enviar(false); };
  </script>
</body></html>

```

### 3. Onde mora o enunciado NOVO

O HTML deve conter **apenas a interação**: tabuleiro, controles, feedback e botões. **Nunca** escreva o enunciado, o título da questão ou as instruções de leitura dentro do jogo — esse texto é redigido separadamente, no campo de enunciado da plataforma. Dentro do jogo cabe, no máximo, um rótulo curto de interface (ex.: "estoque de símbolos") e o feedback reativo de acerto/erro.

### 4. Jogos de dedução: solução única NOVO

Em quebra-cabeças lógicos (quadrado latino, Futoshiki, associação, etc.):

- Garanta **solução única**. Um tabuleiro ambíguo ou impossível invalida a questão.
- De preferência, que seja **resolvível por eliminação**, sem tentativa e erro — assim o aluno raciocina em vez de chutar.
- **Valide por código antes de entregar** (contar soluções por backtracking e checar a resolubilidade por propagação). Não confie na intuição: tabuleiros pequenos enganam.

### 5. Restrições técnicas

- REVISTO Prefira **HTML autossuficiente, sem dependências externas**. É mais robusto e imune a bloqueios de rede no sandbox. Só use CDN se for realmente necessário

— e sempre com versão fixa (ex.: `p5@1.7.0` , nunca `@latest` ), apenas de origens confiáveis (`jsdelivr`, `unpkg`).

- Sandbox `allow-scripts` (sem `allow-same-origin` ). Sem `localStorage` , sem cookies.
- Não use `window.open` , `window.top` nem navegação para fora do `iframe`.
- Largura: 100% do container. Mantenha o jogo **compacto**; a altura do `iframe` cresce sozinha para caber o conteúdo, então ajuste a "altura mínima do `iframe`" perto da altura real para não sobrar faixa branca embaixo.

## 6. Revelar a solução no erro final **NOVO**

Ao esgotar as tentativas ou quando o aluno desistir, mostre o gabarito dentro do próprio jogo *antes* de enviar `acertou: false` . É boa pedagogia: o aluno fecha a atividade entendendo a resposta certa, não apenas sabendo que errou.

## 7. Estilo visual

- Fonte: `-apple-system`, `BlinkMacSystemFont`, `"Segoe UI"`, `Roboto`, `sans-serif` .
- Paleta: roxo `#7e22ce` / verde `#16a34a` (acerto) / vermelho `#dc2626` (erro) / fundo `#faf9f5` .
- Bordas arredondadas (8–12px), transições suaves (~200ms), layout responsivo (grid, fontes 13–16px).
- Para "menos apagado": dê definição com um fundo de painel sob a grade, destaque das pistas fixas e hierarquia clara entre elementos de regra e elementos meramente visuais.

**Honestidade técnica.** **NOVO** A resposta correta vive no JavaScript do cliente, então um aluno avançado consegue lê-la inspecionando o código. Isso é aceitável para prática e aprendizagem, mas estes jogos **não servem para avaliação de alto risco** (provas valendo nota, seleção, etc.).

## 8. Checklist de validação — antes de entregar, verifique cada item

- ☐ O HTML contém a string literal `window.parent.postMessage` em pelo menos um lugar?
- ☐ Esse `postMessage` está dentro de um handler que dispara quando o aluno **termina** o jogo (não no carregamento)?
- ☐ O campo `acertou` é boolean literal ( `true` ou `false` ), não string nem número?
- ☐ O `postMessage` é chamado em **todos** os caminhos de fim de jogo (acerto, erro e desistência)?
- ☐ Existe um caminho terminal que **sempre** dispara o envio, mesmo se o aluno travar? **NOVO**
- ☐ Há uma flag `respondido` que impede envio duplicado?
- ☐ O `meta` usa as chaves padronizadas ( `tentativas` , `tempo` , `resposta` )? **NOVO**
- ☐ Em jogo de dedução: a solução é única e (de preferência) resolvível sem chute, validada por código? **NOVO**
- ☐ O enunciado ficou **fora** do HTML (só a interação dentro do jogo)? **NOVO**

- ☐ Você **renderizou e olhou** o jogo: todos os elementos aparecem, são clicáveis e legíveis no tamanho final? **NOVO**
- ☐ No erro final, o jogo revela a solução antes de enviar `acertou: false`? **NOVO**
- ☐ O HTML começa com `<!DOCTYPE html>` e está completo (head + body)?

Se qualquer item falhar, reescreva antes de entregar. O contrato garante que a *resposta* chega; ele não garante que o jogo é *jogável* — por isso o item de conferência visual.

## 9. Entrega

---

Devolva apenas o HTML completo, sem comentários extras antes ou depois. Não use markdown nem cercas de código. Comece com `<!DOCTYPE html>` e termine com `</html>`.

## 10. Descrição do jogo que você vai criar

---

(descreva aqui o jogo desejado)

Exemplos de descrição: "Aluno arrasta 3 frutas para a balança; acerta se colocar a maçã antes da uva." · "Quiz de 4 alternativas sobre fotossíntese; correta: oxigênio." · "Quadrado latino 4×4 de dedução, solução única, com 4 casas dadas."